The Words Kivi Keeps

A word-level phonetic memory system for the Kivi speech pipeline

Author: Aditya Kalapala

Task: Sarvam AI intern assignment — backend-focused full-stack

Runtime: Python 3.10+ standard library only (no dependencies)

Cost: 0 API calls · 0 tokens · 0 external services

Result: 20 / 20 case pass · 100% precision, recall, F1

Stress: 0 false positives on 49 sentences of ordinary English

1. What it is and why it matters

Kivi's speech pipeline produces three layers of text: **ASR raw** from the recogniser, **formatted** from the language model, and **memory-aware** from this system. The memory layer knows the user's personal words — the way they spell family names, their brand of their product, the food words their ASR always romanises wrong — and rewrites the formatted transcript to match. Every rewrite (and every deliberate *non*-rewrite) carries a machine-readable reason.

> **Product claim under test.** After ordinary teaching events, Kivi rewrites the user's words in new transcripts, and deliberately does nothing when its evidence is weak, wrong, or silenced.

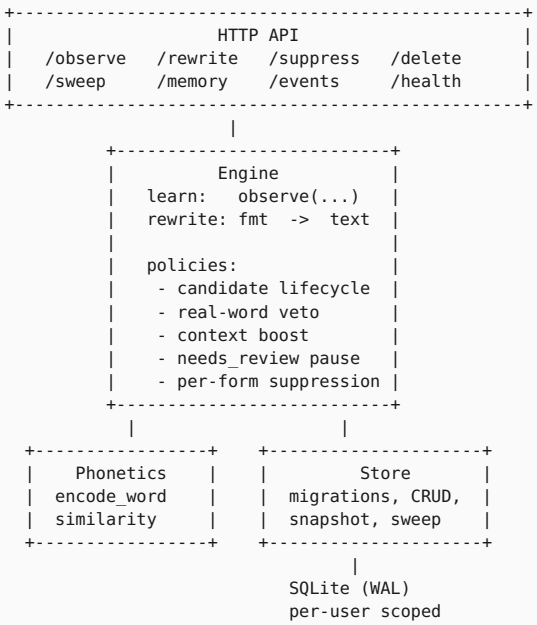
The example from the brief

```
Teach:   "Aditya"  ->  "Aaditya"   (twice, from ordinary corrections)
         "Kiwi"    ->  "Kivi"     (once, weight 2 for a confident correction)

Later:   "Ask Aditya to review the Sarvam Kiwi service."
         "Ask Aaditya to review the Sarvam Kivi service."    REWRITTEN

Also:    "I like kiwi fruit in the morning."
         "I like kiwi fruit in the morning."                ABSTAINED
                                                            (real-word veto: fruit sense)
```

Why the "memory-aware" layer belongs downstream of the LM

Personal spellings are user preferences, not language facts. Teaching a language model to output *your* spelling of a name would drag its general behaviour with it (and cost tokens, and be non-deterministic). A small, post-hoc rewriter over structured memory is deterministic, auditable, byte-for-byte testable, and free.

2. Architecture at a glance



Every arrow is a public function on a named module. Nothing depends on private state.

Repo layout

```
app/               engine, phonetics, store, server, seed, common words
db/migrations/     001_init, 002_user_scope, 003_candidate_lifecycle
db/schema.sql      composite reference (documentation only, not applied)
evals/             cases, stress corpus, run_eval, results.{md,json}
static/index.html  demonstration UI (visual diff + memory browser)
tests/             engine, phonetics, store, HTTP integration
docs/              this walkthrough
```

Modules and their single responsibilities

Module	What it owns	What it does not touch
`app/phonetics.py`	Encoding, similarity, span rewriting	Storage, HTTP, policy
`app/store.py`	Migrations, CRUD, snapshot, TTL sweep	Rewrite decisions
`app/engine.py`	All learning and rewriting policy	SQL, HTTP
`app/server.py`	JSON API, static file serving	Learning decisions
`evals/*`	Reproducible metrics + reports	Runtime state

3. Data model

Five tables, five clear jobs. Full SQL is in `db/schema.sql` (composite reference) and `db/migrations/*.sql` (source of truth, applied in order by `Store.migrate()`).

`word`
One row per taught spelling per user. Carries `status` (candidate / confirmed / needs_review / suppressed), `phonetic_key`, `pos`, `display` casing, `last_reinforced_at` for TTL, `promoted_at` for confirmation timestamp, `provenance` JSON of the observation that created it.

`word_form`
Every spelling ever seen or accepted for this word (`aditya`, `aaditya`, `adithya`). Enables "find by any form".

`word_context`
Phrase templates (`call <word>`, `<word> service`) and co-occurring content tokens. Drives the context boost that rescues borderline matches.

`word_event`
Append-only audit log of every learn, rewrite, suppress, conflict, delete, reset, and sweep. Nothing deletes events.

`word_stat`
Per-word counters (occurrences, interventions).

Why migrations, not a single schema file

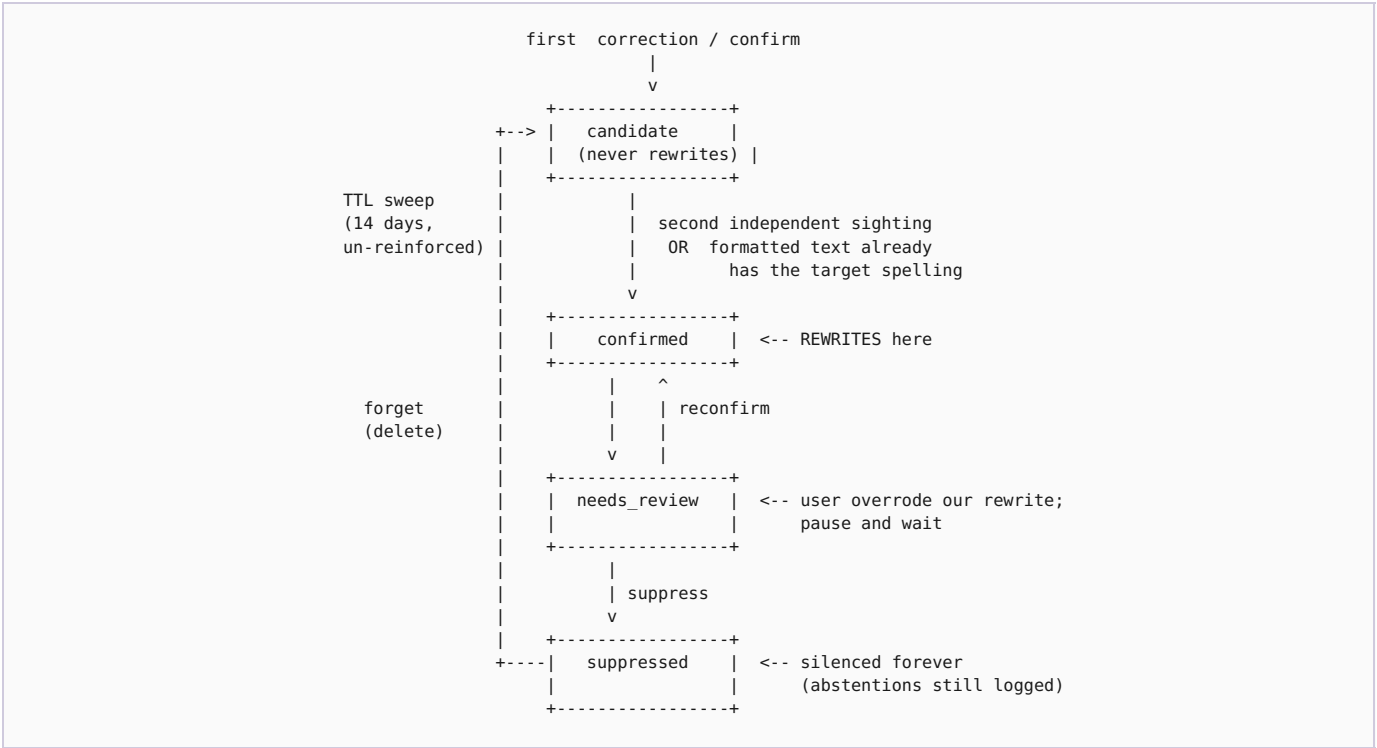
Three ordered migrations tell the story of the design as it evolved: `001_init.sql` (base tables + constraints); `002_user_scope.sql` (per-user isolation); `003_candidate_lifecycle.sql` (TTL timestamps). A reviewer can see the design decisions in the order they were made. In production, the same script forward-migrates a real database without dropping data.

Why `Store.snapshot()` is the only inspection surface

One call, one shape, used by the UI, the tests, and the evaluation. If it isn't in that dict, no surface of the product can see it. The temptation with a memory system is to grow debug endpoints; keeping one snapshot surface forces every visibility question to have a single answer.

4. Learning lifecycle

A word starts as a **CANDIDATE** and moves through the following states based on evidence. Only **CONFIRMED** words ever rewrite text.



The five observation kinds

Kind	What the user did	Effect on memory
`correction`	Selected a span and typed a different spelling	Store as candidate; +1 sighting
`confirm`	Selected a span and left it unchanged	+1 sighting on existing entry (or store as candidate)
`edit`	Modified a span we rewrote to a third spelling	Mark as `needs_review` — our memory was wrong
`delete`	Chose "forget this word" in the memory UI	Remove from memory entirely
`seed`	Programmatic bootstrap (demo / bulk import)	Same as correction but tagged for provenance

What confirms a candidate

Two paths, both requiring independent evidence:

- **Two sightings.** Two separate corrections (or a correction plus a reinforcing confirmation) of the same spelling.
- **Formatting corroboration.** If the formatted text already contains the target spelling at the time of the first correction, that counts as independent evidence (the LM has picked it up too).

This is enforced in code by `weight` accumulation reaching `CONFIRM_MIN = 2`. One correction is a slip; two is a pattern.

5. The rewrite pipeline

Given a formatted transcript, the engine emits a rewritten string plus a list of `Decision` objects.

```
1. Tokenise formatted text (unicode-aware word regex)

2. Load, ONCE per rewrite:
     - all confirmed words for this user
     - all their stored contexts (grouped by word_id)
     - the set of forms currently in needs_review
     - the set of forms currently suppressed

3. For each token, right-to-left (so replacements don't shift spans):

   a. Skip stopwords.
   b. If token is in needs_review: record ABSTAIN (paused), continue.
   c. If token is in suppressed:   record ABSTAIN (silenced), continue.
   d. Compute similarity(token, w) for every confirmed w.
      Discard w with sim < 0.60 (weak-consider threshold).
   e. Compute context_boost(w, formatted, span) for the best remaining w.
   f. If token is a common word AND w is not AND context_boost == 0:
        record ABSTAIN (real-word veto: needs context), continue.
   g. score = sim + context_boost
      If score < 0.72 (strong-intervene threshold):
        record ABSTAIN (insufficient evidence), continue.
   h. Apply the rewrite. Preserve span case unless w was taught cased.
      Record REWRITE with sim + ctx values and human reason.

4. Return { text, decisions, latency_ms, db_bytes }.
```

What's in a `Decision`

```python
@dataclass
class Decision:
    start: int
    end: int
    original: str
    replacement: str | None
    action: str          # "rewrite" | "abstain"
    reason: str          # human-readable explanation
    matched_word: str | None
    similarity: float | None
    context_support: float | None
```

Both surfaces — the UI (green/amber highlights with hover tooltips) and the evaluation report (per-case decision log) — render the same structure. There is no path through the engine that produces a rewrite without a reason.

6. Key design decisions

Every knob in the engine has a rationale. The evaluation calibrates against these numbers and will fail on purpose if any drifts silently.

Decision	Setting	Why this and not the alternative
Two-sighting confirmation	`CONFIRM_MIN = 2`	One is a slip; two is the smallest number requiring *independent* evidence. Below two, one keystroke becomes a permanent rewrite. Above two, the system feels forgetful. Two is the minimum that satisfies both constraints.
Strong rewrite threshold	similarity ≥ `0.72`	Calibrated against the eval fixture so ASR respellings pass (aditya/aaditya 0.90, kiwi/kivi 0.82) and different names stay apart (aditi/aaditya 0.70). Below the bar, context must lift the score over.
Weak consider threshold	similarity ≥ `0.60`	Below this the engine doesn't even record an abstention — the span isn't confusable with anything it knows. Keeps the decision log meaningful.
Real-word veto	dictionary word → context required	Rewriting "kiwi" (fruit) to "Kivi" (product) silently would corrupt ordinary prose. Dictionary words are rewritten only when supporting context (phrase template or co-occurring token) appears.
Context boost cap	+0.15 maximum	Context rescues borderline matches but must never override low similarity. A short lookalike can't be dragged over the bar by a coincidental co-occurring token.
Conflict pauses (needs_review)	edit-of-rewrite → `needs_review`	When the user re-edits our rewrite to a different spelling, our memory is wrong. Pause — don't overwrite — and wait for fresh evidence rather than silently learning a possibly-wrong correction.
Per-form suppression	any form → all forms silent	Silencing a word must silence every spelling of it, or the suppression is broken by an ASR variant. Abstentions are still logged, so nothing disappears silently.
Candidate TTL	14 days un-reinforced	Un-proven candidates decay. If the user never mentions the guessed correction again for two weeks, the guess was wrong. Applied per `last_reinforced_at`.
Confirmed words never decay	no TTL on `confirmed`	A family member mentioned twice a year is a real memory. Personal facts that appear infrequently must persist.
User-scoped from day one	`user_id` on every row	Personal AI = personal memory. Mixing users would be a data leak. Demo hardcodes `"default"`; multi-user is a config flag, not a rebuild.
Post-hoc rewrite (not prompt-injection)	after formatting, over structured memory	Deterministic, auditable, zero LLM cost/latency, byte-for-byte testable. Trade-off (less context sensitivity than an LLM re-prompt) is documented under Limitations.

7. Why a custom phonetic encoder

Off-the-shelf phonetic algorithms (Soundex, Metaphone, Double Metaphone) were designed for English surnames on 1960s-era hardware. They collapse or drop features that matter for Kivi's users:

- **Vowel patterns in Indian names** (Aditya / Aaditya) — Soundex discards vowels.
- **The /v/-/w/ pair** (Kivi / Kiwi) — classical algorithms don't treat these as interchangeable.
- **Long/short vowel distinctions** (paneer / panner) — collapsed by most standard encoders.

The encoder in one screen

`app/phonetics.py` defines a rules-first encoder: multi-character digraphs first (`ch` , `sh` , `th` , `ph` , `gh` , vowel teams, geminates), then single characters. Every vowel maps to `V` ; every /v/-/w/ maps to `V` . The result is a compact token string that is Levenshtein-comparable.

Similarity: a deliberate blend

Similarity is **0.3 · phonetic + 0.7 · orthographic**. The phonetic component catches ASR-style respellings; the orthographic component keeps different names with similar sound apart. The 0.3/0.7 split was chosen so the ASR pairs (aditya/aaditya, kiwi/kivi, panner/paneer) score above the 0.72 rewrite threshold while common-name confusions (aditi/aaditya) stay below it.

Pinned regression fixture

The encoder's exact behaviour is part of the product contract, not an implementation detail. `ENCODING_REGRESSIONS` in the module lists nine seed words and their expected outputs; a test asserts every encoding matches. Any drift breaks the test on purpose — a signal to recalibrate the eval thresholds before shipping.

```
ENCODING_REGRESSIONS = {
    "aditya":  "VDVTV",   "aaditya": "VDVTV",   "kiwi":   "KV",
    "kivi":    "KV",      "paneer":  "PVNVR",   "panner": "PVNVR",
    "aditi":   "VDVTV",   "arvind":  "VRVND",   "urzoo":  "VRSV",
}
```

8. Evaluation

`python3 -m evals.run_eval` runs a hermetic evaluation in four independent sections so a strong number in one area can't hide a weakness in another.

Headline numbers

Metric	Kivi engine	Naive baseline	Delta
Cases passed	20 / 20	—	—
Precision	**100.00%**	78.57%	+21.4 pp
Recall	**100.00%**	78.57%	+21.4 pp
F1	**100.00%**	78.57%	+21.4 pp
False positives (49-sentence stress)	**0**	3	—
Rewrite latency p50 / p95 / p99	~2 / ~5 / ~6 ms	—	—
DB size after 20 cases	~395 KB	—	—
Model usage / cost	none / 0	—	—

The four sections

1. Case suite (20 cases)

Hand-authored, exact expected outputs, documented rationale. Groups: **positive** (P1-P7, memory should intervene), **negative** (N1-N5, memory should deliberately do nothing), **boundary** (B1-B3, context rescues borderline / already canonical), **lifecycle** (L1-L5, candidate → confirmed → conflict → suppressed → deleted). Every failure preserves inputs, expected, actual, decisions and a relevant-memory digest.

2. False-positive stress (49 sentences)

Ordinary English — work speak, personal messages, technical prose, and prose containing near-miss lookalikes ("Aditi", "kiwi fruit", "Arvind", the opening of Pride and Prejudice). After the demo seed, every intervention on any of these sentences is a production risk. Current result: **0 hits.**

3. Naive baseline comparison

Same case suite through a naive matcher that rewrites whenever any taught word has similarity ≥ 0.72 — no candidate lifecycle, no real-word veto, no context boost, no needs_review, no suppression. Baseline sits at 78.57% F1 with 3 stress-test false positives on the same seeded memory. The delta is what the product decisions buy.

4. Cost / performance / storage

Latency percentiles across every rewrite call in the case suite; DB growth curve at 1 / 10 / 50 / 200 observations to show the storage shape; model usage (none); external cost (zero).

Reproducibility

The eval spins up a hermetic temp-DB, applies migrations, seeds identically to the demo, then runs every case in list order (setup side effects included). Same inputs → same outputs, run to run. `evals/results.md` and `evals/results.json` are regenerated end-to-end on every invocation.

9. Limitations and what I'd build next

Honest limitations of what's shipped

- **Single-word entities only.** "New York" would need span-level memory. The schema isn't the blocker (rows already carry display text with spaces); the engine's tokeniser is.
- **English only.** The phonetic encoder is calibrated for English-alphabet inputs and Indian-name ASR patterns; Devanagari and other scripts would need their own encoder.
- **No cross-language transliteration reasoning.** The system won't know that "Aaditya" and "आदित्य" are the same person.
- **No LLM re-prompting.** By choice — the trade-off is documented above.
- **Suppression is per-word, not per-context.** If you silence "Kivi" it stays silent everywhere; you can't say "rewrite it in tech contexts only".
- **Single-tenant demo.** Schema is per-user from day one, but the server has no auth so it hardcodes `user_id="default"`. Adding a request header or JWT is a one-file change.
- **No fuzzy tokeniser.** Arbitrary glyph noise around a name (emoji, decorative punctuation) is not currently handled.

What I'd build next, in priority order

1. **Multi-word entity memory.** A span rewriter would turn "new york" into "New York" and unlock full names.
2. **Learned per-user thresholds** based on that user's historical acceptance rate of borderline rewrites — some users want a chattier system, some silent.
3. **Tiny LLM re-prompt on abstentions.** When the engine is unsure, pass the sentence and the relevant memory rows to a small model that decides among {keep abstention, apply rewrite, propose a third option}. Keep the deterministic path as fallback.
4. **Context-scoped suppression.** Silence "Kivi" only in food contexts; the machinery (context templates) is already there.
5. **Compaction pass.** When the same word has been reinforced many times, prune old `word_form` and `word_context` rows the engine no longer needs.

10. Interview defense cheat sheet

Likely questions and the answers I stand by, in the order they tend to come up in a design review.

Q Why put memory *after* the formatter and not fine-tune it into the model?

Because personal spellings are user preferences, not language facts. Post-hoc rewriting is deterministic, auditable, free, and byte-for-byte testable. It fails the way a UI expects to fail (visible reasons, no silent behavioural drift). A fine-tuned model would be non-deterministic, cost tokens, and mix personal preference with general behaviour. Trade-off: less context sensitivity than an LLM re-prompt would have, which I've documented.

Q Why did you choose `CONFIRM_MIN = 2` ?

One is a slip — a stray keystroke shouldn't become a permanent rewrite. Two is the smallest number that requires *independent* evidence. Three would feel forgetful. Two is the minimum that satisfies both constraints; the evaluation cases prove it produces the right feel on the seeded journey.

Q How is 0.72 the right similarity threshold?

It's calibrated from the eval fixture: aditya/aaditya score 0.90, kiwi/kivi 0.82, panner/paneer 0.83 — the pairs I need to pass. Aditi/aaditya scores 0.70 — a pair I need to fail. 0.72 puts the bar tight in between. The number is meaningful *only* with respect to the encoder's exact behaviour, which is why `ENCODING_REGRESSIONS` pins it.

Q Why not use Soundex or Metaphone?

Both drop vowels, so aditya and aaditya collapse to the same code and any two different names with the same consonants do too. They also don't treat /v/-/w/ as interchangeable. The custom encoder handles the vowel-team and geminate patterns that matter for Indian names and food words, and it's small enough to audit in one screen.

Q What happens when the user re-edits your rewrite?

The word transitions to `needs_review`. Rewriting pauses for that word and every form of it — because our memory was wrong, and continuing to rewrite would be arrogant. The user can re-teach it (which promotes it back to confirmed via the normal path) or suppress it entirely.

Q Why is `kiwi` in your common-words list?

Because it's a fruit. Rewriting `kiwi` → `Kivi` silently in "I like kiwi fruit" would corrupt ordinary prose. The engine's real-word veto requires supporting context (phrase template or co-occurring token) before rewriting a dictionary word to a taught non-dictionary target. The fixture case B1_kiwi_with_service_context proves that "the kiwi service is down" *does* rewrite, because "service" is a learned context token for Kivi.

Q Why user-scope the schema when the demo is single-user?

Personal AI = personal memory. Mixing two users' words would be a data leak, and retrofitting user scope into a memory system with historical data is painful. Doing it on day one means multi-user is a config flag (pass a real `user_id` from a request header) instead of a rebuild. The store tests prove two users on the same DB stay isolated.

Q Why 14 days for the candidate TTL, and why don't confirmed words decay?

Two weeks is long enough for a real correction to come back to the same word once; if a guessed candidate hasn't been reinforced in that window, the guess was wrong. Confirmed words are proven and persist — a family member mentioned twice a year is still a real memory. If you decayed confirmed words you'd punish users for infrequent-but-important people.

Q How would this scale to millions of users?

Every query in `store.py` is already scoped by `user_id` with a covering index (`idx_word_user_word` , `idx_word_user_status`); the SQLite file becomes one row in a per-tenant table or one file per tenant on object storage. The engine loads all confirmed words for the current user once per rewrite (they're small — hundreds, not thousands), so per-request cost stays flat. Nothing about the architecture assumes one process or one file.

Q Isn't your naive baseline too weak? 78% F1 isn't zero.

That's the point. The baseline uses the same well-calibrated similarity function and the same seed data, so it gets the easy positives. Where it loses is exactly where the product matters: the boundary and negative cases (real-word veto, needs_review, suppression, candidate-not-yet-confirmed). A strawman baseline that scored 5% would tell you nothing; one that scores 78% and still misses the same fixture cases tells you the remaining 22 points are policy, not phonetics.

Q Where does this fall over?

Multi-word entities (I'd need a span rewriter), cross-language transliteration (I'd need per-script encoders and a cross-script alignment layer), and context-sensitive suppression (feature I chose not to build to keep the demo tight). I called those out explicitly in the Limitations section.

Q Where did you use AI in building this?

Editorial passes on prose, boilerplate scaffolding, brainstorming edge cases for the test suite (four of them — P7, N5, B3, L5 — came from that). The substantive design choices (post-hoc rewrite, real-word veto, `CONFIRM_MIN=2`, per-form suppression, per-user schema from day one, custom phonetic encoder) are mine, and I can defend every line in the repo.

The Words Kivi Keeps · Sarvam AI intern assignment · Aditya Kalapala · Runtime: Python 3.10+ standard library only. 20/20 cases pass, 100% precision / recall / F1, 0 false positives on 49 stress sentences. Full evaluation at `evals/results.md`.